Final Project Report

Student(s): | Lukas Ernst-Ove Roos – lura23@student.bth.se

1. Project Idea

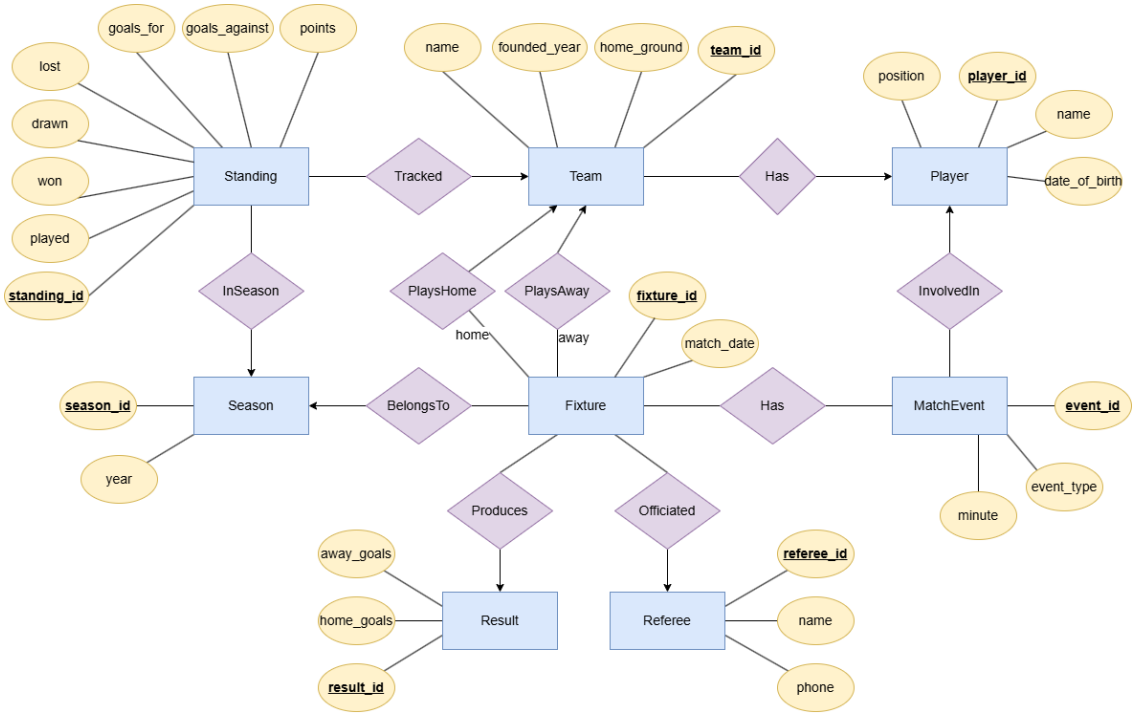
For this assignment I have designed and implemented a Sunday League Manager system. The problem being solved is that amateur football leagues are typically managed using spreadsheets or paper, which is error-prone and difficult to maintain over time.

The main users of the system will be league administrators and team managers who need to schedule fixtures, record match results and track standings and player statistics throughout a season.

The application allows users to: register teams and players, schedule fixtures, record match results, view the league standings table, view top scorers and view detailed match reports. The system automatically updates standings when a result is recorded using a database trigger. The data was generated artificially using a seed script generated by AI, populating the database with 8 teams, 80 players, 4 referees, 28 fixtures and corresponding results and match events.

2. Schema Design

The database consists of 8 tables derived from the Entity-Relationship diagram, designed using classic E/R notation with rectangles for entity sets, ovals for attributes and diamonds for relationships. All entities use a surrogate integer ID as the primary key.



Season (season_id, year) - simplified to just a year value since amateur seasons map directly to a calendar year and do not require explicit start and end dates.

Team (team_id, name, home_ground, founded_year) - stores information about each team in the league.

Player (player_id, name, date_of_birth, position, team_id) - the team_id foreign key links each player to their current team. *ON DELETE SET NULL* is used so that a player record is not lost if a team is deleted.

Referee (referee_id, name, phone) - stores referee contact information.

Fixture (fixture_id, match_date, home_team_id, away_team_id, season_id, referee_id) - represents a scheduled match. Contains two foreign keys to the Team table for home and away teams, reflecting the two relationship diamonds (PlaysHome and PlaysAway) in the E/R diagram.

Result (result_id, fixture_id, home_goals, away_goals) - kept separate from Fixture so that a fixture can exist before a result is known. A *UNIQUE* constraint on fixture_id enforces the one-to-one relationship between Fixture and Result.

MatchEvent (event_id, fixture_id, player_id, event_type, minute) - records individual events such as goals, yellow cards, red cards and substitutions. A single table with an event_type column was chosen over separate tables for each event type to keep the schema simple. *ON DELETE CASCADE* ensures events are removed if a fixture is deleted.

Standing (standing_id, team_id, season_id, played, won, drawn, lost, goals_for, goals_against, points) - stores pre-computed standings per team per season. A *UNIQUE* constraint on (team_id, season_id) ensures one row per team per season. Standings are

automatically updated by a trigger when a result is inserted, avoiding the need to recalculate on every query.

3. SQL Queries

Query 1 – League Standings (JOIN + aggregation)

This query retrieves the standings table for a season ordered by points then goal difference. It joins Standing and Team to display team names alongside statistics.

```
set @season = 1;
select t.name, s.played, s.won, s.drawn, s.lost,
    s.goals_for, s.goals_against,
    (s.goals_for - s.goals_against) as goal_difference,
    s.points
from standing s
join team t on s.team_id = t.team_id
where s.season_id = @season
order by s.points desc, goal_difference desc;
```

Query 2 – Top Scorers (multi-table JOIN + GROUP BY)

This query counts goal events per player by joining three tables: MatchEvent, Player and Team. Results are grouped by player and ordered by goals scored descending.

```
select p.name, t.name as team, count(*) as goals
from matchevent me
join player p on me.player_id = p.player_id
join team t on p.team_id = t.team_id
where me.event_type = 'goal'
group by p.player_id, p.name, t.name
order by goals desc;
```

Query 3 – Fixtures for a Team (multi-table JOIN)

This query retrieves all fixtures for a specific team. It joins Fixture to Team twice using aliases (ht for home team, at for away team) and uses *LEFT JOIN* to Result so unplayed fixtures appear with a null score.

```
set @team = 4;
select f.match_date,
    ht.name as home_team,
    r.home_goals,
    r.away_goals,
    at.name as away_team
```

```
from fixture f
join team ht on f.home_team_id = ht.team_id
join team at on f.away_team_id = at.team_id
left join result r on f.fixture_id = r.fixture_id
where f.home_team_id = @team or f.away_team_id = @team
order by f.match_date;
```

Query 4 – Trigger: Update Standings After Result

This trigger fires automatically after a new row is inserted into the Result table. It retrieves the home and away team IDs from the Fixture table, then updates the Standing table for both teams incrementing played, won, drawn, lost, goals and points as appropriate. This ensures standings are always kept consistent without manual updates.

```
delimiter //
create trigger after_result_insert
after insert on result
for each row
begin
    declare home_team int;
    declare away_team int;

    select home_team_id, away_team_id
    into home_team, away_team
    from fixture
    where fixture_id = new.fixture_id;

    -- update home team standing
    update standing
    set played = played + 1,
        goals_for = goals_for + new.home_goals,
        goals_against = goals_against + new.away_goals,
        won = won + if(new.home_goals > new.away_goals, 1, 0),
        drawn = drawn + if(new.home_goals = new.away_goals, 1, 0),
        lost = lost + if(new.home_goals < new.away_goals, 1, 0),
        points = points + case
  when new.home_goals > new.away_goals then 3
            when new.home_goals = new.away_goals then 1
            else 0 end
        where team_id = home_team and season_id = (
```

```
  select season_id from fixture where fixture_id =
new.fixture_id
        );

    -- update away team standing
    update standing
    set played = played + 1,
        goals_for = goals_for + new.away_goals,
        goals_against = goals_against + new.home_goals,
        won = won + if(new.home_goals < new.away_goals, 1, 0),
        drawn = drawn + if(new.home_goals = new.away_goals, 1, 0),
        lost = lost + if(new.home_goals > new.away_goals, 1, 0),
        points = points + case
  when new.away_goals > new.home_goals then 3
           when new.away_goals = new.home_goals then 1
           else 0 end
        where team_id = away_team and season_id = (
  select season_id from fixture where fixture_id =
new.fixture_id
        );
end //
delimiter ;
```

Query 5 – Procedure: Match Report

This stored procedure takes a fixture ID and returns two result sets: the match result with team names, date and referee, and all match events ordered by minute. It is called from the Python application using *cursor.callproc('get_match_report', [fixture_id])*.

```
drop procedure if exists get_match_report;
delimiter //
create procedure get_match_report(in p_fixture_id int)
begin
    -- result
    select ht.name as home_team,
r.home_goals,
          r.away_goals,
```

```
          at.name as away_team,
          f.match_date,
          ref.name as referee
from fixture f
    join team ht on f.home_team_id = ht.team_id
    join team at on f.away_team_id = at.team_id
    left join result r on f.fixture_id = r.fixture_id
    left join referee ref on f.referee_id = ref.referee_id
    where f.fixture_id = p_fixture_id;

    -- match events
    select me.minute, me.event_type, p.name as player, t.name as
          team
    from matchevent me
    join player p on me.player_id = p.player_id
    join team t on p.team_id = t.team_id
    where me.fixture_id = p_fixture_id
    order by me.minute;
end //
delimiter ;
call get_match_report(4);
```

4. Discussion and Resources

The project was implemented as a console-based Python application using the mysql-connector-python library. All queries are written as explicit raw SQL strings in accordance with the assignment requirement of not using ORM.

The Standing table uses a pre-computed approach rather than calculating standings dynamically. This was a deliberate design choice to improve read performance and to allow the trigger to demonstrate real-time updates to the database state.

The Python application provides 9 menu options covering all main features. The stored procedure is called directly from Python using *cursor.callproc()*, demonstrating integration between the application layer and the database. In addition to the five core queries, the Python application implements supporting functions including adding new fixtures, viewing all fixtures, listing all teams and listing all referees. These provide a complete console interface for managing the league without needing to interact with the database directly.

Source code: [github link]

Changelog

Person	Task	Date
Lukas	Decided on the project idea	2026-05-04
Lukas	Designed the E/R diagram	2026-05-04
Lukas	Set up the server environment	2026-05-06
Lukas	Wrote the SQL schema	2026-05-06
Lukas	Wrote the SQL queries and seed data	2026-05-06
Lukas	Implemented the Python console application	2026-05-09
Lukas	Added to the Python console application	2026-05-10
Lukas	Wrote the project report	2026-05-10
Lukas	Set up Git repository	2026-05-10